



Polymarket: Deposit Wallet Upgrade

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

May 21, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Emergency Recovery Suite Missing Native ETH Support	4
3.1.2	<code>DepositWalletFactory</code> upgrades can silently change <code>BEACON</code> immutable, desyn- chronizing wallet logic from actual dispatch	4
3.2	Low Risk	5
3.2.1	Migrated beacon-forwarded proxies incorrectly remain ERC-1822 proxiable	5
3.2.2	Batch Execution Accepts Undocumented Signature Format	6
3.2.3	Ownership Handover Signatures Can Front-Run Cancellation Or Be Reused	6
3.2.4	Batch Execution Discards Detailed Revert Reasons	7
3.2.5	Repeated Pause Calls Reset the Recovery Timelock	7
3.2.6	Session Signer Authorization Lacks Validation	8
3.2.7	Factory Initializer Lacks Zero-Address Validation	8
3.2.8	Unbounded <code>returndata</code> bytes copy in <code>DepositWallet.execute()</code> enables griefing of the operator	8
3.3	Informational	9
3.3.1	Factory Uses Unnamed Solady Role Constants	9
3.3.2	Numerous instances of deprecated <code>@solidity memory-safe-assembly</code> comment annotation should be updated	9
3.3.3	<code>DepositWalletBeacon.optOut()</code> behavior should be clarified in documentation . .	10
3.3.4	<code>_authorizeImplementation()</code> logic can be inlined within <code>authorizeImplementation()</code>	10
3.3.5	UUPS to Beacon proxy migration assumptions not sufficiently documented in <code>deploy()</code>	11
3.3.6	Inaccurate developer comment copied from Solady	11

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From May 5th to May 7th the Cantina team conducted a review of [deposit-wallet](#) on commit hash [28643fdb](#). The team identified a total of **16** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	8	5	3
Gas Optimizations	0	0	0
Informational	6	5	1
Total	16	12	4

2.1 Scope

The security review had the following components in scope for [deposit-wallet](#) on commit hash [28643fdb](#):

```
src
├── DepositWallet.sol
├── DepositWalletBeacon.sol
├── DepositWalletFactory.sol
├── Errors.sol
├── Events.sol
├── dev
│   ├── TestHelper.sol
│   └── Vm.sol
├── libraries
│   ├── EIP712Helpers.sol
│   ├── ExchangeHelpers.sol
│   ├── ExchangeLib.sol
│   ├── Ownable.sol
│   ├── SessionSignerLib.sol
│   ├── WalletHelpers.sol
│   └── WalletLib.sol
├── migration
│   └── BeaconForwarder.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Emergency Recovery Suite Missing Native ETH Support

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: The wallet has a dedicated "emergency exit hatch" for when it is paused, allowing the owner to directly withdraw ERC-20 and ERC-1155 tokens after a timelock. However, there is no equivalent function for withdrawing native ETH.

The wallet can definitely hold native ETH (it inherits Solady's `Receiver`), and it can spend it during normal operation. If the operator (Polymarket) were to become unavailable and the owner needed to use the emergency exit hatch to recover funds, any native ETH in the wallet would be stuck until the operator returned or the owner could find another way to relay a signed batch.

Recommendation: Add a `withdrawETH(address to, uint256 amount)` function to the `onlyPaused` recovery suite.

Polymarket: Fixed in [PR 55](#).

Cantina Managed: Verified. A new paused exit hatch for native tokens has been added.

3.1.2 `DepositWalletFactory` upgrades can silently change `BEACON` immutable, desynchronizing wallet logic from actual dispatch

Severity: Medium Risk

Context: `DepositWalletFactory.sol#L119-L120`, `DepositWallet.sol#L206-L208`, `DepositWallet.sol#L393-L395`, `DepositWallet.sol#L417-L419`, `DepositWallet.sol#L425-L428`

Description: `DepositWalletFactory` declares `BEACON` as a construction time immutable that is intended to resolve the implementation for every newly deployed `DepositWallet`. However, because it is an immutable embedded in implementation bytecode rather than a value stored in proxy storage, it is only stable for a given factory implementation. A future UUPS upgrade can silently change the value returned by `BEACON()` for the factory proxy by upgrading to a new implementation compiled with a different beacon address.

This is problematic because `DepositWallet._beacon()` resolves the beacon dynamically via `DepositWalletFactory(factory()).BEACON()`. This is only correct if the factory's `BEACON()` return value remains permanently aligned with the actual dispatch beacon of every wallet that references that factory. A factory upgrade that changes `BEACON` therefore means existing newly deployed beacon and migrated legacy proxies still dispatch through the *original* beacon stored in their ERC-1967 beacon slot (`LibClone::deployDeterministicERC1967BeaconProxy`) and `BeaconForwarder` respectively, while `DepositWallet` runtime code will start using the *new* beacon for critical operations such as `optOut()` / `optIn()` and the session-signer guard in `execute()`.

This creates a disconnect in which `DepositWallet` logic no longer targets the same beacon that actually governs the wallet's dispatch. In practice, this means an owner might think they are opting out of upgrades, but they would be writing that state to the wrong beacon, leaving their wallet still subject to the original beacon's defaults.

Furthermore, the session signer guard, which is intended to prevent session signers from directly calling the beacon, will now block the new beacon but allow calls to the original implementation. A compromised session signer could then force a wallet to opt-in or opt-out on the original beacon, bypassing the owner-only paused/timelock requirement.

Recommendation: Do not allow the factory-reported beacon to diverge from the beacon already bound to existing wallets.

One approach is to move the factory's immutable to proxy storage and have `_authorizeUpgrade()` explicitly reject any new factory implementation whose `BEACON()` value differs from the currently installed factory implementation's `BEACON()`. If the system ever needs to replace the beacon contract itself, it should be handled as an explicit migration with a new factory or a coordinated wallet migration path, not as an ordinary factory implementation upgrade.

Alternatively, consider refactoring the logic such that beacon resolution does not depend on the factory's current implementation-level immutable. If resolution must work for both newly deployed beacon proxies and migrated legacy UUPS proxies without calling the factory, the wallet needs context-aware beacon discovery.

This could be achieved by first attempting to read the proxy's ERC-1967 beacon slot: if non-zero, this is a newly deployed proxy, so that slot is the real beacon; otherwise, read the proxy's ERC-1967 implementation slot, and if non-zero, then this is a legacy UUPS proxy migrated through the forwarder, so `BEACON()` should be callable on the implementation.

Polymarket: Fixed in [PR 56](#).

Cantina Managed: Verified. Beacon resolution is now decoupled from the factory. `DepositWallet._beacon()` reads the ERC-1967 beacon slot when present, and otherwise falls back to reading the ERC-1967 implementation slot as a `BeaconForwarder` and calling its `BEACON()` getter.

3.2 Low Risk

3.2.1 Migrated beacon-forwarded proxies incorrectly remain ERC-1822 proxiable

Severity: Low Risk

Context: [BeaconForwarder.sol#L31-L41](#)

Description: `BeaconForwarder.proxiableUUID()` is currently callable both on the implementation itself and through any legacy ERC-1967 proxy that has migrated to `BeaconForwarder`. This violates the standard UUPS safety invariant described by OpenZeppelin: a proxy that points to a proxiable contract must not itself appear proxiable, because doing so can cause another proxy to upgrade to that proxy address and brick itself by recursively delegating until out of gas.

That risk exists here after migration. The migration path legitimately relies on a direct `staticcall` to the `BeaconForwarder` implementation address so legacy wallets can pass the ERC-1822 compatibility check during `upgradeToAndCall()`. However, once a legacy wallet has migrated, it is still an ERC-1967 proxy, now pointing at `BeaconForwarder`. Because `proxiableUUID()` does not reject `delegatecall` context, the migrated proxy also returns the ERC-1967 implementation slot and therefore looks like a valid ERC-1822 implementation target.

As a result, if any UUPS upgrade later accepts that migrated proxy address as an implementation target, the upgrade check can succeed even though the target is a proxy rather than an implementation. A proxy upgraded to that address would then delegate into another proxy, which reads its own implementation slot and re-enters itself recursively until the call runs out of gas, bricking the proxy. Even though the current migration authorization flow only intends to approve the shared `BeaconForwarder` implementation address, leaving this behavior in place creates a latent footgun if a migrated proxy address is ever accepted as an implementation target.

Recommendation: Consider adding a `notDelegated`-style guard to `BeaconForwarder.proxiableUUID()` so it reverts when invoked through a proxy and only succeeds when called directly on the implementation contract. This change does not break the migration flow, because the migration compatibility check calls `proxiableUUID()` directly on the `BeaconForwarder` implementation address, not through the proxy. It only prevents migrated proxies from presenting

themselves as valid ERC-1822 implementations and removes the possibility of bricking a proxy if such a proxy address is ever accepted as an upgrade target.

Polymarket: Fixed in [PR 61](#).

Cantina Managed: Verified. A guard has been added to prevent migrated proxies from appearing as valid ERC-1822 targets by having `proxiableUUID()` revert when executed in a `delegatecall` /proxy context.

3.2.2 Batch Execution Accepts Undocumented Signature Format

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `execute` function is designed to take an EIP-712 signature for a `Batch` struct. However, because it routes the signature through the general-purpose `isValidSignature` function (which inherits Solady's ERC-1271 logic), it also accepts signatures wrapped in `PersonalSign`.

Specifically, if a user signs the batch digest using the "Ethereum Signed Message" (`PersonalSign`) prefix, the wallet will accept it. While this isn't a security vulnerability because the signature still has to come from the owner and still commits to the exact same batch data, it is an undocumented "ghost" feature. It can lead to confusion for integrators and UI developers who expect only direct EIP-712 signatures to work, and it might hide intent if a user is prompted with an opaque hex string instead of a readable typed-data structure.

Recommendation: If `PersonalSign` support is intended, it should be documented and tested. If not, the `execute` function should use a more direct verification path that only accepts the specific `Batch` EIP-712 signature format.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.2.3 Ownership Handover Signatures Can Front-Run Cancellation Or Be Reused

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The two-step ownership handover process uses EIP-712 signatures from the pending owner to complete a transfer. However, these signatures only commit to the new owner and a deadline; they do not include a nonce or handover identifier. `completeOwnershipHandover` is also permissionless, so anyone holding the pending owner's signature can submit it while the same `pendingOwner` is active and the deadline has not passed.

This weakens cancellation. If the current owner decides to cancel a handover after the pending owner's signature has been released, the pending owner or any third party that obtained the signature can front-run the cancellation with `completeOwnershipHandover`, transferring ownership before `pendingOwner` is cleared.

Even if the cancellation lands first, the old signature is only deactivated while there is no matching `pendingOwner`. If the owner later re-initiates handover to the same address before the signature deadline, the original signature becomes valid again, can complete the new handover immediately, and can be submitted by anyone. This can transfer ownership sooner or at a different time than the owner or pending owner expects.

Recommendation: Include a nonce or unique handover ID in the `OwnershipHandover` struct that is signed. The value should change whenever a handover is initiated or canceled, ensuring that each signature only authorizes one specific handover attempt.

The current owner should also set the handover deadline when initiating the transfer (not the pending owner) and the wallet should store that deadline with the pending handover. Completion should verify the pending owner's signature against the stored owner-chosen deadline, rather than allowing the pending owner to choose an arbitrary deadline in the signed message. Keep the deadline as an expiry bound, but do not rely on it as a revocation mechanism.

Polymarket: Fixed in [PR 57](#).

Cantina Managed: Verified. The current owner is now allowed to set the handover acceptance deadline and the pending owner must now also sign over a per-handover nonce.

3.2.4 Batch Execution Discards Detailed Revert Reasons

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: When a wallet executes a batch of calls and one of those calls fails, the wallet reverts with a generic `CallFailed()` error. While this correctly maintains the atomicity of the batch (the whole transaction fails), it throws away the actual revert data returned by the target contract.

This makes debugging failed batches significantly harder for operators and users. Instead of seeing a helpful message like "Insufficient balance" or "Invalid price," they only see `CallFailed()`, requiring them to use advanced tracing tools to figure out what actually went wrong.

Recommendation: Capture the revert data from the failed call and bubble it up, or include it in a more descriptive custom error like `BatchCallFailed(uint256 index, address target, bytes reason)`.

Polymarket: Fixed in [PR 60](#).

Cantina Managed: Verified. The error has been made more descriptive as recommended.

3.2.5 Repeated Pause Calls Reset the Recovery Timelock

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `pause()` function in `DepositWallet` can be called even if the wallet is already paused. Every time it is called, it overwrites the `paused` timestamp with the current time.

This is an issue because all emergency recovery functions (like `withdrawERC20`) require the wallet to have been paused for at least the duration of the `timelockDelay`. By calling `pause()` again, the owner (or an automated script) accidentally resets the clock, forcing the owner to wait the full delay all over again. This could delay emergency fund recovery in a stressful situation.

Recommendation: Make the `pause()` function idempotent. It should check if the wallet is already paused and, if so, return early without updating the timestamp.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.2.6 Session Signer Authorization Lacks Validation

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The `authorizeSessionSigner` function allows the owner to authorize the zero address as a session signer, or to set a `validUntil` timestamp that is already in the past.

While these "dead" authorizations don't create a security hole (they are simply unusable), they still emit success events and pollute the contract's state. This can be confusing for off-chain monitoring systems or indexers that track active session signers, as they will see "Authorization" events that never actually granted any usable power.

Recommendation: Add validation to ensure `_sessionSigner != address(0)` and `_validUntil > block.timestamp`.

Polymarket: Fixed in [PR 60](#).

Cantina Managed: Verified. The recommended validation has been added.

3.2.7 Factory Initializer Lacks Zero-Address Validation

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The `DepositWalletFactory.initialize` function does not check if the `_owner` or `_admin` addresses are set to `address(0)`. If the factory is initialized with a zero owner, it becomes "bricked" for governance actions. No one would be able to add/remove admins, authorize implementations for migration, or (most importantly) upgrade the factory itself in the future.

While this is a deployment-time configuration error rather than a runtime exploit, the lack of a simple safety check makes it easier to accidentally lose control of the protocol's central factory.

Recommendation: Add `require` statements to the `initialize` function to ensure that neither the owner nor the admin can be the zero address.

Polymarket: Fixed in [PR 60](#).

Cantina Managed: Verified. The recommended validation has been added.

3.2.8 Unbounded `returndata` bytes copy in `DepositWallet.execute()` enables griefing of the operator

Severity: Low Risk

Context: [DepositWallet.sol#L221](#)

Description: `DepositWallet.execute()` performs arbitrary external calls and captures the full return-data into memory:

```
(bool success, bytes memory result) =  
    _batch.calls[i].target.call{value: _batch.calls[i].value}(_batch.calls[i].data);
```

By assigning `bytes memory result` to the second return value, the EVM copies the target's entire `returndata` into the wallet's memory before the subsequent `require(success, CallFailed())` and event emission. A malicious target can return an extremely large payload, causing excessive memory expansion and gas consumption in the wallet. Since operators are the entities that route user-signed

batches from the factory to arbitrary targets, this creates a griefing vector where an otherwise valid batch can be made to fail or become prohibitively expensive, consuming the operator's entire gas limit.

Recommendation: Avoid decoding arbitrary and potentially unbounded `returndata` bytes by capping the length copied from external calls.

Polymarket: Acknowledged, we simulate transactions so this gets handled offchain.

Cantina Managed: Acknowledged.

3.3 Informational

3.3.1 Factory Uses Unnamed Solady Role Constants

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The `DepositWalletFactory` uses Solady's default role constants `_ROLE_0` and `_ROLE_1` directly in the code for the Admin and Operator roles. While the contract comments explain that `_ROLE_0` maps to Admin and `_ROLE_1` maps to Operator, the actual functions (like `isAdmin`, `isOperator`, `addAdmin`, etc...) use the generic Solady names.

This isn't a security vulnerability because the roles are applied consistently, but it makes the code harder to read and maintain. Developers and auditors have to keep the mapping in their heads, which increases the chance of a mistake if more roles are added in the future.

Recommendation: Define descriptive constants at the top of the contract and use them throughout:

```
uint256 internal constant ADMIN_ROLE = _ROLE_0;
uint256 internal constant OPERATOR_ROLE = _ROLE_1;
```

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.3.2 Numerous instances of deprecated `@solidity memory-safe-assembly` comment annotation should be updated

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: There are numerous instances of assembly blocks using the legacy comment-form memory safety annotation:

- `src/libraries/SessionSignerLib.sol:24` .
- `src/libraries/EIP712Helpers.sol:21` .
- `src/DepositWalletBeacon.sol:116` .
- `src/DepositWalletBeacon.sol:133` .
- `src/DepositWalletBeacon.sol:150` .
- `src/DepositWalletBeacon.sol:158` .
- `src/DepositWalletBeacon.sol:171` .
- `src/DepositWalletBeacon.sol:188` .

- `src/DepositWalletBeacon.sol:216` .

These blocks do not appear incorrect from a memory-safety perspective, but the comment-form annotation is deprecated and scheduled for removal in favor of the explicit block annotation syntax.

Recommendation: Replace all instances of:

```
/// @solidity memory-safe-assembly
assembly {
  //...
}
```

with:

```
assembly ("memory-safe") {
  //...
}
```

Polymarket: Fixed in PR 60.

Cantina Managed: Verified. The deprecated annotations have been replaced.

3.3.3 `DepositWalletBeacon.optOut()` behavior should be clarified in documentation

Severity: Informational

Context: `DepositWalletBeacon.sol#L224-L226`

Description: If the default beacon implementation has changed, then calling `DepositWalletBeacon.optOut()` again will bump the pinned implementation even if the wallet has not first called `DepositWalletBeacon.optIn()` . This effectively allows wallets to upgrade to the latest version while still not allowing automatic beacon upgrades. If this behavior is intentional, then it should be explicitly documented as it is not strictly idempotent.

Recommendation: Consider updating the inline documentation accordingly.

Polymarket: Fixed in PR 60.

Cantina Managed: Verified. This behavior is now explicitly documented.

3.3.4 `_authorizeImplementation()` logic can be inlined within `authorizeImplementation()`

Severity: Informational

Context: `DepositWalletFactory.sol#L308-L317`

Description: The logic contained within `_authorizeImplementation()` was previously shared between the initializer and `authorizeImplementation()` . Given that it is no longer present or needed within the initializer, this functionality can be simply inlined within `authorizeImplementation()` .

Recommendation: Inline the legacy logic.

Polymarket: Fixed in PR 60.

Cantina Managed: Verified. The logic has been inlined.

3.3.5 UUPS to Beacon proxy migration assumptions not sufficiently documented in `deploy()`

Severity: Informational

Context: `DepositWalletFactory.sol`#L205-L207

Description: The dev comment present in the `DepositWalletFactory.deploy()` Natspec is true only for newly-deployed beacon proxies. A legacy wallet with a given ID can already exist at one address, and the upgraded factory can still deploy a fresh beacon proxy with the same ID at a different address because the proxy template changed and therefore the deterministic address derivation changed.

This also means that the address returned by `predictWalletAddress(id)` is not the address of an existing legacy wallet that was later migrated to `BeaconForwarder`.

Recommendation: Clearly document these assumptions within the developer documentation.

Polymarket: Fixed in [PR 60](#).

Cantina Managed: Verified. The distinction between upgraded legacy wallets and new beacon deployments is now explicitly documented.

3.3.6 Inaccurate developer comment copied from Solady

Severity: Informational

Context: `DepositWalletBeacon.sol`#L13

Description: Some of the developer comments present in `DepositWalletBeacon` have been copied verbatim from Solady. The reference to `LibClone.deployERC1967BeaconProxy` is outdated, since it is actually `deployDeterministicERC1967BeaconProxy()` that is now used within the factory.

Recommendation: Update the developer comment to correctly reflect the current implementation.

Polymarket: Fixed in [PR 60](#).

Cantina Managed: Verified. The comment has been corrected.